

Fetching data from JIRA

I have added the code for fetching the data from the JIRA. I have mentioned the prompts below and added screenshots for the same. On a gist, I split this requirement into 4 modules and described the prompts to generate the output.

1. UI implementation to connect to JIRA
2. Backend implementation to get the details stored and hit the end point.
3. UI implementation to show all the user stories fetched from JIRA as a select button.
4. On selecting a user story data has to be profiled in the test case generator.

Note: Acceptance criteria was not recieved in the response payload. Hence it was not prefilled.

PROMPTS and SCREENSHOTS

Do you have good understand of this code base? Both front end and backend code

I'll quickly inspect both backend and frontend source files so I can answer accurately how well I understand the codebase and identify any gaps.

I'm starting with a focused read of core entry points plus repository memory, so I can assess architecture coverage instead of giving a generic answer.

I've confirmed the main request flow is wired end-to-end; next I'm reading the remaining backend/frontend files to verify data contracts, prompt logic, and where responsibilities split.

I now have the core flow and schemas; I'm checking the LLM client plus remaining app bootstrap files to ensure there aren't hidden behaviors I'm missing.

Yes. I have a solid working understanding of both sides of this codebase.

What I understand now:

1. End-to-end flow
 - The React UI collects a user story form and calls backend APIs for either test-case generation or feature-file generation in App.tsx and api.ts.
 - The backend exposes two routes, validates inputs with Zod, calls Groq, and returns structured responses in generate.ts and generateFeature.ts.
2. Backend architecture
 - Server setup, CORS, health route, route mounting, and global error handlers are in server.ts.
 - Prompt templates and prompt builders are centralized in prompt.ts.
 - Request/response contracts are defined in schemas.ts.
 - Groq integration (including model/base URL/env handling and completion call) is in groqClient.ts.
3. Frontend architecture
 - Single-page React app with local state for form, loading, errors, and expandable test-case rows in App.tsx.

- Typed API calls and response handling in api.ts with shared TS interfaces in types.ts.
- App bootstrap is minimal in main.tsx.
- 4. Tooling and runtime
- TypeScript + Express backend scripts in package.json.
- Vite + React frontend scripts in package.json.

If you want, I can next give you:

1. A quick “risk review” (likely bugs, edge cases, and security concerns).
2. A map of exactly where to add new features.
3. A prioritized refactor plan for maintainability.

As a QA architect, I want to connect with JIRA to fetch the user stories in this test case generator and generate the test cases. I am going to split this requirements as 4 part.

- 1. UI implementation to connect to JIRA**
- 2. Backend implementation to get the details stored and hit the end point.**
- 3. UI implementation to show all the user stories fetched from JIRA as a select button.**
- 4. On selecting a user story data has to be profiled in the test case generator.**

Below is my requirements for point 1

**[MANDATORY]] I want a button in Front end Saying connect to JIRA.
On clicking the button, a dialogbox should open. It should ask for JIRA user's base URL, Email ID and JIRA API Key. Have a button in the dialog box called "View UserStories"**

[STRICT] - Understand the above requirement. And check how this can be implemented

User Story to Tests

Generate comprehensive test cases from your user stories

Story Title *

Description

Please fill in this field.

Acceptance Criteria *

Additional Info

Generate

Generate Feature file

Connect to JIRA

User Story to Tests
Generate comprehensive test cases from your user stories

Story Title *
Enter the user story title...

Description
Additional description (optional)...

Acceptance Criteria *
Enter the acceptance criteria...

Additional Info
Any additional information (optional)...

Connect to JIRA
Enter your JIRA credentials to fetch user stories.

JIRA Base URL *
https://your-domain.atlassian.net

Email ID *
qa.user@company.com

JIRA API Key *
Enter JIRA API key

Cancel View UserStories

Generate Generate Feature file Connect to JIRA

[MANDATORY] Collect the data from the user in the dialog box and store it as base URL, Email ID and API Key in the below API when user clicks the View Userstory button

[MANDATORY] This data has to be passed in the endpoint and the authorisation.

Endpoint URL: {base-url}/rest/api/3/search/jql Example:
https://sreeabinayak.atlassian.net/rest/api/3/search/jql

Authorisation header should contain {email id } as username and {api_key} as password.

Below is the request body.

```
{
  "jql": "project = UserStories AND issuetype = Story",
  "fields": [
    "key",
    "summary",
    "description",
    "status",
    "reporter"
  ]
}
```

On hitting the URL, user will get list of user stories. Say 25 user stories.

Please plan what changes are required for passing the data to the API and fetching the information.

Connect to JIRA

Enter your JIRA credentials to fetch user stories.

JIRA Base URL *

Email ID *

JIRA API Key *

Fetches 25 user stories from JIRA

Cancel

View UserStories

Name	X	Headers	Payload	Preview	Response	Initiator	Timing
user-stories		General					
		Request URL		http://localhost:8090/api/jira/user-stories			
		Request Method		POST			
		Status Code		200 OK			
		Remote Address		[::1]:8090			
		Referrer Policy		strict-origin-when-cross-origin			

Perfect! Now create a dialog box with the all the user stories fetched from the endpoint. Show only the title. Show these titles as a select button. Only 1 user story should be selectable at a time. Have the UI looks good.

What all the changes required for this? Please plan it accordingly.

JIRA User Stories

25 stories

Select a user story to load into the test generator.

☐	US-25	Employee Profile Employee Self
☐	US-24	Ahc Reports Reports Ahc
☐	US-23	Secretary Dashboard Doctor Calender
☐	US-22	Masters Others Approval Setup
☐	US-21	Employee Profile Basic Details
☒	US-20	Doctor Dashboard Op Dashboard
☐	US-19	Reports Admin Reports Others
☐	US-18	Consultant Dashboard Mv Appointments

Cancel

Use Selected Story

JIRA User Stories

25 stories

Select a user story to load into the test generator.

- ☐ US-8 Radiology Transactions Technicians Worklist
- ☐ US-7 Doctor Activities Raise Request
- ☐ US-6 Nurse Activities Raise Request
- ☐ US-5 Billing Execution Deposit Refund
- ☐ US-4 Reports View Reports Others
- ☐ US-3 Daycare Nurse Activities Raise
- ☒ US-2 Wards Dashboard Nurse Activities
- ☐ US-1 Consultant Dashboard My Inpatients

Cancel

Use Selected Story

Good UI! Once a Test case is selected, on clicking the “Use Selected story” button prefill the title in theStory title field, description in the description field in the test case generator.

Story Title *

Consultant Dashboard My Inpatients

Description

As a user, I want to consultant dashboard my inpatients so that the related functionality works as expected.

Acceptance Criteria *

Enter the acceptance criteria...

Additional Info

Any additional information (optional)...

Generate Generate Feature file Connect to JIRA